

ScoutAI: A Multi-Agent Job-Search Platform on LangGraph

Architecture, Engineering Decisions, and Their Costs

Sai Nikhil Kunapareddy

September 19, 2026

Abstract

Scout is a multi-agent platform that runs job-search assistants as Slack direct-message bots. An agent is a declarative specification — a system prompt and a set of tools — compiled onto a single shared LangGraph model/tool loop and served to many users from one process, on any of three interchangeable model backends. The platform covers 29 companies through 16 source modules that share no upstream API, tailors every search to the user’s résumé through a two-stage graph whose cache *is* its checkpointed state, and scopes one agent entirely to the companies where the user has a referral.

This report describes the architecture and, more importantly, the decisions behind it and what each one cost: why conversation history lives in a checkpoint thread rather than in application state; why an unreachable job board must never be reported as an empty one; why company-to-board routing is a lookup table rather than a prompt; why a model retry belongs inside the graph node and not around the turn; and why a turn that runs out of tool hops must repair its own thread before apologising. It closes with the system’s limits and the specific constraint each next step would remove.

Contents

1	Problem and scope	2
2	The system at a glance	2
3	Layering: three seams	3
4	The two graphs	4
5	The life of a message	5
6	The job-source subsystem	6
7	Decision log	8
7.1	Why the retry lives in the model node (#9)	8
7.2	Why one process holds exactly one agent (#8)	8
7.3	Why the referral list is not checkpointed (#6)	8
7.4	Why capability is split rather than prompted (#7)	9
8	Failure modes	10
9	Concurrency and state	11
10	Observability	12
11	Testing strategy	13

12	Deployment and operations	13
13	Limits, and what the next constraint would be	14
A	Interview cheat sheet	16
B	The sixty-second version	17

1 Problem and scope

An AI/ML job search looks like a generation problem and is almost entirely a data integration problem. The roles worth seeing are spread across a few dozen company careers boards that share no API and no common format: some publish a JSON endpoint that their own single-page app consumes, some publish RSS and nothing else, and some publish only server-rendered HTML. None of them rank results against a particular résumé, and none of them know that the person searching has a former colleague at Stripe — which is, in practice, the single strongest signal about whether an application goes anywhere.

Scout was built against six requirements:

- R1. Tailored.** Every search is shaped by the user’s own résumé, without the user restating it.
- R2. Referral-aware.** A role at a company where the user knows someone outranks a marginally better-matched role where they know nobody.
- R3. Scope-complete.** One agent answers “what is open where I could ask for a referral” and, when it cannot check something, says so rather than returning a quietly short list.
- R4. Unprompted.** A digest arrives every morning without anyone typing anything.
- R5. Cheap.** It runs on an AWS free-tier t4g.micro (2 vCPU, 916 MB, no swap).
- R6. Portable across models.** A hosted Claude outage, a rate limit, or a cost spike must not be a dead bot; a local model must be one message away.

Explicit non-goals: applying on the user’s behalf, multi-tenant SaaS, and a web front end.

Slack DMs were chosen as the entire interface because a DM is three things at once — an input channel, a notification channel, and an identity — so R4 costs nothing extra and no front end has to be written, authenticated, or hosted. The cost of that choice is a 3,500 character practical message limit and no rich interaction beyond text, which is why replies are split on line boundaries and never on characters (a hard split would break a job link).

2 The system at a glance

Figure 1 is the whole system on one page. Table 1 is the agent catalogue and Table 2 the model backends. In numbers: 6,265 lines of Python in the package, 5,593 lines of tests, 389 tests, 100 % statement *and* branch coverage enforced in CI across Python 3.10–3.13, running as six systemd units on one machine.

Table 1: The four registered agents. Adding one is a single file plus one registry line; nothing about the graph changes.

Key	Name	Tools	Résumé-tailored	In daily digest
bigtech	BigTech Agent	clock, location, referrals_read, 14 job-source modules covering 27 companies	yes	yes
edu	Edu Agent	clock, location, referrals_read, Northeastern, Boston University	yes	yes
referral	Referral Window	clock, referrals (read <i>and</i> write), search_referral_jobs	no	yes
resume	Resume Parser	get_resume_profile	n/a	no

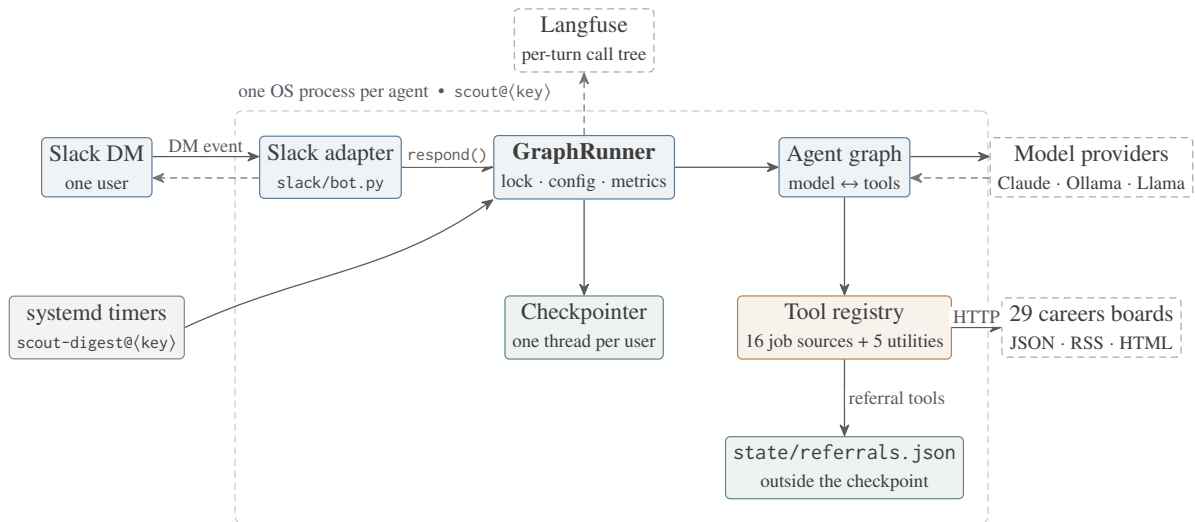


Figure 1: System context. Everything inside the dashed boundary is one Python process holding one Slack app’s token; a second agent is a second process, not a second code path. Two things cross the boundary in both directions — model calls and Slack messages — and two only leave it: job-board HTTP and observability.

Two things in Table 1 are load-bearing rather than incidental. First, the job agents get `referrals_` read and the Referral Window gets `referrals`: only one agent may write the list, because an agent with `add_referral` within reach will eventually record something mid-search that the user never asked for. Second, “résumé-tailored” and “in digest” are separate flags even though three of four agents would be satisfied by one. The Referral Window is the case where they differ: it searches a *scope* rather than a résumé, and it still has something to report every morning.

Table 2: Model backends. All three are LangChain chat models, so the graph binds tools and sends messages identically to each; history is stored as provider-agnostic message objects, which is what allows a switch mid-conversation.

Backend	Integration	Why it exists	Failure mode
anthropic	ChatAnthropic, Claude Opus 5	Default. Reliable multi-hop tool use, which the digest sweep depends on.	Needs a key; falls back to Ollama for one turn.
ollama	ChatOllama, local	Offline development and the fallback target; no key, no cost, no egress.	Not running ⇒ fails on first request.
llama	ChatOpenAI pointed at Meta’s compatible endpoint	A second hosted provider without a second integration to maintain.	Needs a key; no bespoke client to debug.

The third row is a small decision worth naming: Meta serves an OpenAI-compatible endpoint, so the OpenAI integration is used as the client rather than writing a bespoke one. Adding a provider is one builder function plus one dictionary entry, and nothing else in the system learns that a provider was added.

3 Layering: three seams

The architecture is organised around three interfaces, chosen so that each *kind* of change lands in exactly one place (Figure 2, Table 3).

Seam 1 — ConversationalAgent (an ABC).

Everything the Slack adapter is allowed to know: `respond`, `reset`, `set_backend`, `backend_label`, `last_backend`, `tool_names`. It is implemented *once*, by `GraphRunner`, and inherited by both the plain agent and the résumé-tailored pipeline — so the adapter has no branch for “is this the complicated one”.

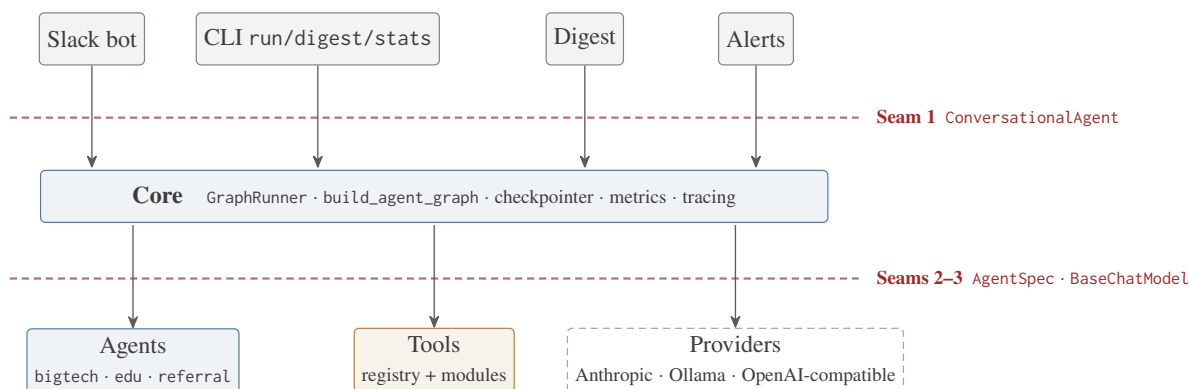


Figure 2: The three seams. Above the top seam nothing knows what a graph is; below the bottom seam nothing knows what Slack is. The core is the only layer that has seen both, and it is the layer that almost never changes.

Seam 2 — AgentSpec (a dataclass).

An agent is declared, not coded: key, display name, system prompt, tool modules, default backend, and the two behaviour flags. The rule that makes the seam real is that adding an agent must never edit the graph.

Seam 3 — the LangChain chat model.

One builder per provider behind BaseChatModel. The consequence reaches much further than provider swapping: because history is stored as provider-agnostic message objects rather than as any vendor’s wire format, a user can change model in the middle of a conversation and the thread replays cleanly into the new provider.

Table 3: Change surface. The test of the layering is not how the diagram looks but how small the diff is for each routine change.

Change	What it touches	Never touches
Add an agent	one file in scout/agents/, one line in AGENTS	the graph, the Slack layer
Add a tool	one module with register(reg), one entry in a spec’s tool_modules	the registry, the graph
Add a job source	one module under tools/jobs/, one line in directory.SOURCES	the shared HTTP, render and relevance code
Add a company on an existing platform	one line in that platform’s BOARDS (it reaches the referral directory on its own)	everything else
Add a model provider	one builder, one entry in _BUILDERS	agents, tools, history
Add a daily digest	set in_digest on the spec, add one systemd timer	scout/digest.py

4 The two graphs

Every agent compiles to the same model/tool loop (Figure 3). Agents that opt into résumé tailoring are wrapped in a second, outer graph (Figure 4).

Window trimming is load-bearing. The tail of the conversation is selected with `trim_messages(..., start_on="human")`. That argument is not cosmetic: providers reject a conversation that opens on the assistant’s side, and a tool result must never lead the window or be separated from the tool call it answers. Because tool traffic is checkpointed — unlike the hand-rolled loop this replaced, where it was per-turn —

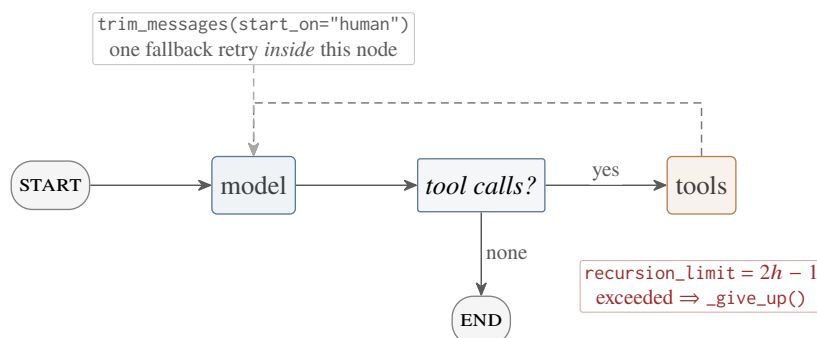


Figure 3: The shared agent graph. Three mechanisms hang off it, and each is deliberately placed *in* a node rather than around the turn: window trimming and the fallback retry live in the model node, and the hop limit is expressed as LangGraph’s recursion limit.

the model remembers what it already looked up, at the cost of a shorter reach back through a tool-heavy conversation.

The hop limit is arithmetic, not a guess. A tool hop is two LangGraph super-steps (model, then tools) and a turn always ends on a model step, so h hops is $2h - 1$ steps. Exceeding it raises `GraphRecursionError`, which is handled rather than propagated (§8).

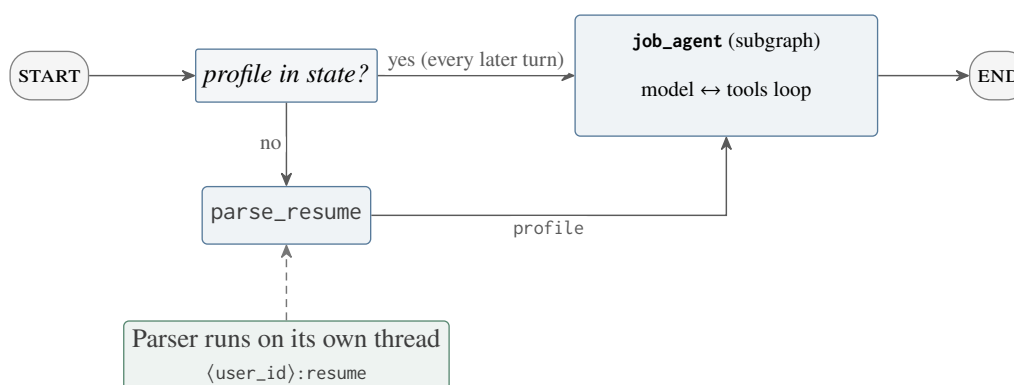


Figure 4: The résumé-tailored orchestration graph. The router reads the profile out of checkpointed graph state, so the cache *is* the state — not a dictionary kept beside it. The parser’s own tool calls and JSON reply never enter the job agent’s history; only the rendered brief crosses, appended to the job agent’s system prompt.

Three properties follow from putting the cache in graph state rather than in a Python dictionary. It survives a restart whenever `CHECKPOINT_DB` is set, so a deployed bot parses a résumé once rather than once per restart. It is dropped by `--reset` along with the thread, which is exactly the behaviour a user expects after editing their résumé. And it needs no invalidation logic, because there is no second place for it to go stale in.

One subtlety worth knowing before an interviewer finds it: a subgraph is handed the recursion limit *afresh*, so the outer graph’s extra nodes raise a *floor* (`_min_steps`) rather than adding to the limit. Adding would silently hand the inner tool loop extra hops.

5 The life of a message

Figure 5 traces one DM end to end. The ordering of three steps matters more than the rest: the per-user lock is taken before anything else, the traced config is a *copy*, and metrics are recorded after the reply is known but inside the lock.

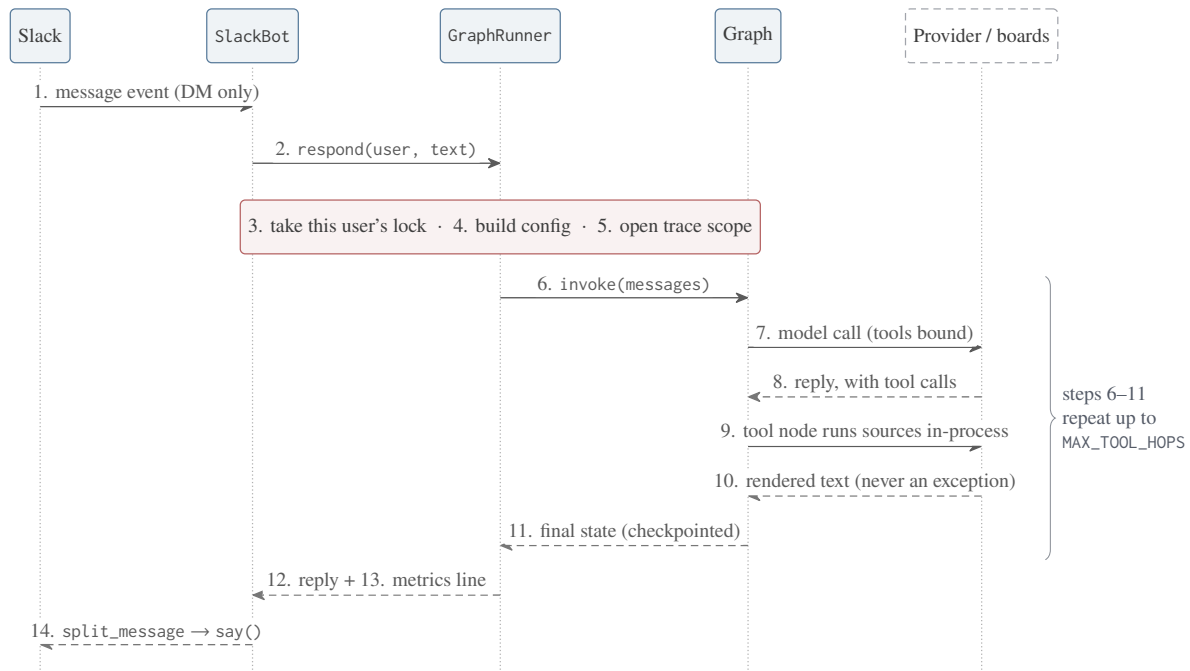


Figure 5: One turn, end to end. Steps 6–11 are the loop of Figure 3; everything outside them happens exactly once per message. Step 9 runs tools in-process — the tool node is Python, not a service call — which is why a tool must return a sentence rather than raise.

The lock. slack-bolt dispatches events on a thread pool, so two messages from the same user can arrive concurrently and interleave into one thread. One RLock per user serialises that user’s turns while different users still run in parallel — the granularity that matters, given that a turn can take tens of seconds.

Why the traced config is a copy. GraphRunner._config returns an *untraced* config, because that same config is used for checkpointer reads and for the repair written after a stuck turn, neither of which is part of the turn a human would want to read in a trace. tracing.traced yields a new dictionary with the callback added. Nothing in the graph branches on whether a turn is observed.

6 The job-source subsystem

Sixteen source modules cover 29 companies. They are organised along two axes at once (Figure 6): by *platform*, which is how an implementation has to be organised, and by *company*, which is how a user thinks.

Every source exposes its search twice. A registered tool returning Slack text, and a search(keywords, limit) -> list[JobPosting] | None matching a Searcher protocol, of which the tool is a thin renderer. The second form is what lets one caller search many companies and merge the results; without it, the Referral Window would have to drive a dozen tools and re-parse their prose.

None is not []. Every fetch helper returns None for any failure whatsoever — connection refused, HTTP 500, a body that will not decode — and that None propagates all the way to the reply as “I could not look”. An empty list means “I looked, and there is nothing”. Collapsing the two is the one failure mode that sends someone away from a company they should have asked, and it is the single contract every source in the system is written against.

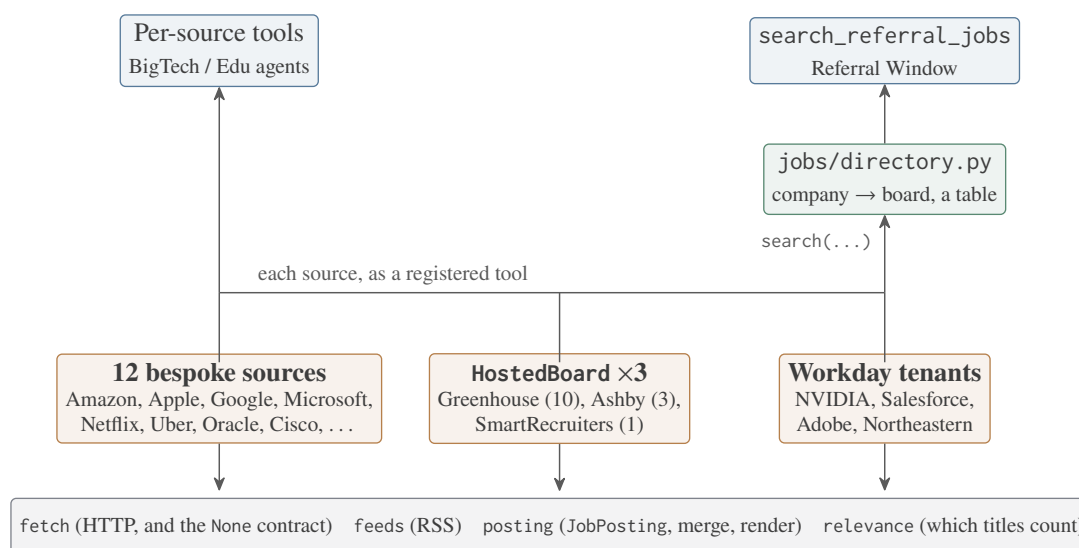


Figure 6: Two indexes over one set of sources. Upward-left: every source is a registered tool a job agent calls by name. Upward-right: every source is also a plain `search(...)` function, which `directory.py` indexes *by company* so the Referral Window can sweep a list of employers rather than a list of tools. Everything rests on one shared layer.

A keyword box is not a title filter. Several boards match the query against the whole posting, so searching “machine learning” returns account executives whose description mentions it. Every title is therefore re-gated through `relevance.is_ai_ml_role` after the source returns, and when the model supplies no keywords the module falls back to five profile queries merged and de-duplicated.

Table 4: The sixteen source modules and what each had to work around. “Date” records whether the source publishes a real timestamp; where it does not, the source’s own wording is carried verbatim in `posted_label` and never coerced into a date.

Source	Mechanism	Date	The awkward part
Amazon	JSON endpoint	yes	No relevance filter; each profile query is run and merged.
Apple	server-rendered page	yes	The documented search API needs a CSRF token; the page’s own embedded state is read instead.
Google	HTML scrape	no	No API and no dates, so the site’s own ordering is kept.
Microsoft	Eightfold JSON	yes	Rows are buried under data; the old careers API is gone.
Netflix	Eightfold JSON	yes	Keyword, location and recency filtering are all server-side.
Uber	JSON board	yes	One request returns the whole US board.
Oracle	Oracle Cloud Recruiting	yes	Rows sit under <code>items[0]</code> ; a missing key means <i>unreachable</i> , not empty.
Cisco	Phenom widget POST	yes	Rows under <code>refineSearch.data</code> .
Bloomberg	Avature scrape	no	<code>careers.bloomberg.com</code> is a redirect dead end; nothing is published.
Lenovo	Avature RSS	label	Feed carries relative wording only.
Boston University	SilkRoad RSS	label	No search API at all.
Greenhouse	HostedBoard	yes	10 companies, one shape; no server-side filtering.
Ashby	HostedBoard	yes	3 companies, same shape.
SmartRecruiters	HostedBoard	yes	1 company; country field double-checked.
Workday	CXS POST	label	3 tenants; 20-row page cap, and an out-of-range offset <i>wraps</i> to page one, so paging cannot terminate.
Northeastern	Workday CXS	label	Same platform, different tenant shape.

One shape, written once. Greenhouse, Ashby and SmartRecruiters are not three sources; they are one shape written three times — a board per company at a predictable URL, no server-side filtering, one JSON row per opening. HostedBoard holds that shape (slug lookup, fetch, title filtering, and the three answers such a tool must tell apart: company we do not host, board that was down, board with nothing matching). A platform module supplies only what differs: where its boards live and how to read one row. Workday is deliberately *not* folded into it: it takes a POST body rather than a slug in a path, and its URL needs three parts rather than one. Nothing about the two lines up, so forcing them together would produce an abstraction that is harder to read than either.

7 Decision log

Table 5 is the short form of every decision that shaped the system, with what it cost. The four expanded below are the ones whose reasoning is least obvious from the code.

7.1 Why the retry lives in the model node (#9)

The obvious place to retry a failed model call is around the turn: catch, switch backend, run the turn again. That is wrong here for a reason specific to graphs. A LangGraph node that raises commits nothing to state, so retrying *inside* the node starts from a clean slate — no half-written assistant message, no orphaned tool call — while every tool result the turn has already fetched is still in state and is not re-fetched. Retrying around the turn would either discard that work or replay it, and replaying means hitting a dozen careers boards a second time. Three tests pin the behaviour: `test_failed_call_is_retried_on_the_fallback`, `test_fallback_sees_no_trace_of_the_failed_attempt`, and `test_tool_results_already_fetched_survive_the_fallback`.

The fallback is disclosed to the user rather than applied silently, because a local 3B model and Claude differ enough that a silent swap would be misleading. An empty `FALLBACK_BACKEND` disables the retry entirely, which is what a container with no Ollama in it needs — otherwise every Claude failure fails twice, slowly.

7.2 Why one process holds exactly one agent (#8)

A process holds one Slack bot token. An earlier version ran all digests in one process, which meant every agent’s morning report was posted by whichever app was the default — three reports stacked in one conversation, attributed to the wrong bot. Splitting them is not a scaling decision; it is an identity one. The systemd template unit makes the split nearly free (§12): `scout@bigtech` and `scout@edu` are the same unit file instantiated twice, each reading its own `EnvironmentFile`. The cost is memory — three resident Python processes plus three periodic ones on a 916 MB box with no swap — which is why the digest timers are staggered ten minutes apart rather than fired together.

7.3 Why the referral list is not checkpointed (#6)

History is disposable. It is trimmed on every turn, it is deleted by `--reset`, and losing it costs the user nothing they would notice. The referral list is the opposite: a handful of companies the user typed once and expects to find again, and the thing that makes one agent’s answer worth reading. Putting it in the checkpointer would couple its lifetime to history’s, so a reset — something a user does casually — would silently delete it.

It is therefore a JSON document in `state/`, rewritten whole under a lock, written to a temporary file and renamed so a crash mid-write leaves the previous list intact rather than a truncated file. A file that will not parse raises rather than silently starting over: the file is the user’s data, and a bad parse is far more likely to be a bug than a real reset. `deploy.sh` excludes `state/` from its `rsync`, so a redeploy cannot overwrite the box’s list with whatever happens to be on a laptop.

Table 5: Decision log. A decision with no cost in the right-hand column is usually a decision that has not been thought about.

#	Decision	Rejected	Why	What it costs
1	LangGraph, not a hand-rolled tool loop	keep the bespoke while loop	Checkpointing, subgraphs and a tool node arrive for free; tool traffic becomes part of history, so the model remembers what it looked up	Tool messages consume the MAX_TURNS window
2	History <i>is</i> the checkpointer thread, keyed by Slack user id	an application-level message list	Provider-agnostic replay, model switching mid-conversation, --reset is a thread delete	Two bots on one database file share a thread unless each gets its own path
3	Résumé profile cached in graph state	a dictionary beside the graph	Survives restarts, needs no invalidation logic, dropped with the thread	A changed résumé needs an explicit --reset
4	None $\neq$ [] in every source	return [] on failure	Makes a completeness claim possible at all	Every source and every caller must carry the distinction
5	Company-to-board routing is a table	let the model route by name	The Referral Window can name what it could <i>not</i> check	The table is maintained by hand
6	Referral list outside the checkpointer	store it with history	User-typed data survives --reset and redeploy	A second store, with its own locking and atomic write
7	Split capability: referrals_read vs referrals	one tool module for both	A searching agent cannot mutate the list	Two modules instead of one
8	One process per agent, one Slack token	one process hosting every agent	Each digest lands in its own DM window	Three resident processes on 916 MB
9	Model retry <i>inside</i> the model node	retry around the whole turn	A node that raises commits nothing, so the retry starts clean but keeps tool results already fetched	The fallback must be a working backend, or every failure fails twice
10	Tools return sentences, never raise	let exceptions propagate	One dead board cannot fail a whole turn	A real bug can hide behind a polite message
11	Observability added to a <i>config</i> , never a code path	instrument the graph	No branch anywhere knows whether a turn is traced	The root span must be owned explicitly to make the trace's input/output the question and the answer
12	Digest membership derived from in_digest	a registry beside AGENTS	A new job agent joins the digest by existing	Each one still needs its own systemd timer
13	settings.py never raises on import	validate configuration at import	The package stays importable and testable with no .env	Validation must be called explicitly at each entry point

7.4 Why capability is split rather than prompted (#7)

The job agents are told in their system prompt never to edit the referral list. That instruction is real, and it is not the mechanism. The mechanism is that `add_referral` and `remove_referral` are not in their tool set at all: they import `referrals_read`, which registers only `list_referrals`. An agent that searches twelve boards and ranks the results will, often enough to matter, decide that recording something would be helpful — and the user never asked it to.

The same principle runs the other way for the Referral Window, which *may* write. What makes that safe is that its only search tool is `search_referral_jobs`, which takes the list as its scope; there is nothing in its tool set to search with off-list. The invariant is guarded directly by `test_the_referral_window_can_only_search_within_the_list`.

Related, and worth stating because it surprises people: a tool that needs to know *who* is asking

declares `*`, `config: RunnableConfig`. LangChain injects it and hides it from the schema the model sees, so the model is never asked to supply a user id and therefore cannot get one wrong. The annotation must stay exactly `RunnableConfig` — widening it to `RunnableConfig | None` stops LangChain recognising it, the injection silently stops, and `config` reappears as a parameter the model is asked to fill. `test_the_user_id_is_not_in_the_schema_the_model_sees` guards it.

8 Failure modes

Every external dependency in this system is somebody else’s website. Degradation is therefore the normal case, not the exceptional one, and each mode has a designed response (Figure 7, Table 6).

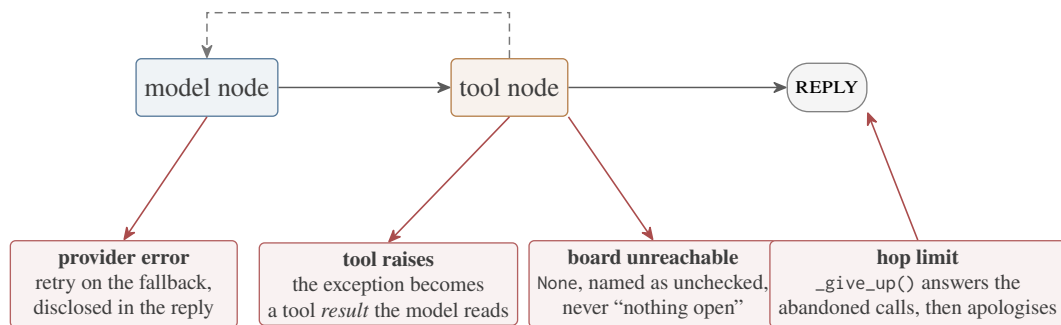


Figure 7: Failure paths, in red. None of them terminates the turn: a failed provider is retried on another, a raised tool becomes text the model can act on, an unreachable board is reported as unchecked rather than empty, and a turn that exhausts its hops repairs its own thread before apologising.

The repair is the interesting one. When a turn runs out of tool hops, the run stops with tool calls still unanswered. Every provider rejects a conversation in which a tool request has no result, so simply apologising would leave a thread that breaks the user’s *next* message, not just this one. `_give_up()` therefore writes a synthetic result for each abandoned call before recording the apology. It is a one-screen method and it is the difference between one bad turn and a conversation that has to be reset.

Table 6: Failure modes and their designed responses. Every row has a test, because a degradation path that is not tested is a degradation path that does not work.

Failure	Detected by	Response	Guarded by
Provider request fails	exception in the model node	Retry once on FALLBACK_BACKEND within the same turn; the reply says which model answered	test_failed_call_is_retried_on_the_fallback
No fallback configured	FALLBACK_BACKEND empty or unknown	Raise, so the Slack layer reports it rather than failing twice slowly	test_an_empty_fallback_setting_disables_the_retry
A tool raises	ToolNode(handle_tool_errors=...)	The exception is turned into a tool result the model reads and can act on	test_failing_tool_becomes_a_message_for_the_model
Model invents a tool name	LangGraph	Reported back to the model as a result, not an error	test_unknown_tool_is_reported_to_the_model
Hop limit exhausted	GraphRecursionError	_give_up() answers every abandoned tool call, <i>then</i> records the apology, so the thread stays valid	test_thread_survives_the_hop_limit
Job board unreachable	fetch returns None	Named in the reply as unchecked; never merged into “nothing found”	test_an_unreachable_board_is_named_not_read_as_empty
Every board down	all searches None	A different sentence again: “could not reach any of them”	test_every_board_being_down_is_not_reported_as_nothing_open
Company has no board	directory.resolve returns None	Listed as a coverage gap the user must check themselves	test_a_company_with_no_board_is_named_too
Referral store corrupt	JSON decode error	Refuse to read, answer in words; never silently start a fresh list	test_an_unreadable_store_answers_in_words
Model returns nothing	empty reply text	Substitute a readable message rather than sending an empty Slack post	test_empty_reply_becomes_something_readable
Résumé file unreadable	parse failure	Name the file instead of raising; the search degrades to untailored	test_a_corrupt_file_names_the_file_instead_of_raising
Langfuse will not start	exception building the client	Log once, run untraced for the life of the process	test_a_turn_still_answers_when_tracing_will_not_start
Unit exceeds its restart burst	systemd OnFailure=	scout-alert@ DMs the last 20 journal lines	tests/test_alert.py

9 Concurrency and state

Four things are stored, in three places, with three different lifetimes (Figure 8, Table 7). Which one a piece of data lands in is decided by a single question: would the user mind losing it?

Table 7: The four state stores and their lifetimes.

Store	Key	Lifetime	Why it is where it is
Conversation history	Slack user id	thread, dropped by --reset	It is the checkpoint; provider-agnostic messages are what allow a model switch mid-conversation
Résumé profile	(user):resume	same thread lifetime	Caching in graph state removes the need for any invalidation logic
Referral list	owner (Slack user id)	indefinite	Typed by hand once; must outlive a reset and a redeploy
Turn metrics	—	journald retention	One key=value line per turn: greppable, no network, always on

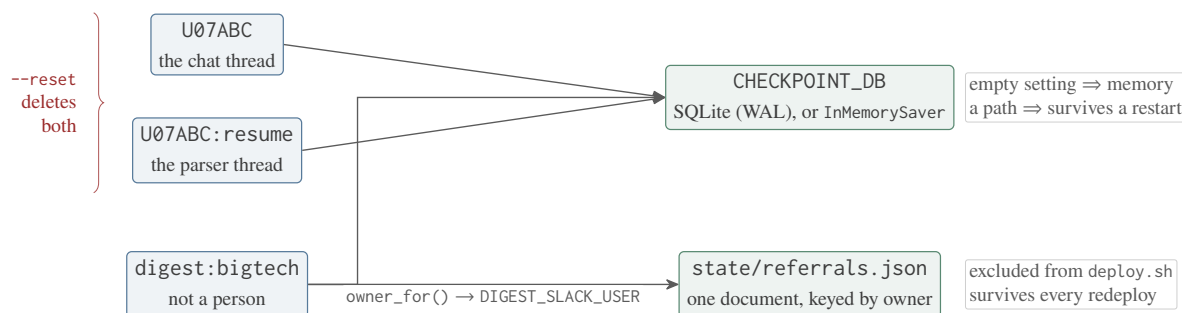


Figure 8: What is stored where, and what a reset destroys. Three thread ids map into one checkpoint database; the referral list sits outside it on purpose. The digest’s thread is not a Slack user, so `owner_for()` maps it onto the person the digest is addressed to — without that, the morning report would look up a user with no referrals and quietly stop ranking by them.

Concurrency is handled at three levels and nowhere else. `slack-bolt` dispatches on a thread pool; one `RLock` per user serialises that user’s turns while other users run in parallel. The referral store takes a single process-wide lock, because the per-user locks order only *one* user’s turns and two users editing at once would read-modify-write over each other. SQLite is opened with `check_same_thread=False` and a 30-second busy timeout, in WAL mode so a reader never blocks the writer — which matters because the digest process reads the same file while the bot is mid-turn.

One consequence is easy to miss and expensive to discover in production: thread ids are the bare Slack user id, not namespaced by agent. Two bots pointed at one `CHECKPOINT_DB` therefore *share* a thread, and your conversations with them interleave into one history. Each interactive agent gets its own database path in its own environment file. The digest was never exposed to this, because its threads are `digest:<key>`.

10 Observability

Three tiers, with deliberately different costs (Table 8). The design rule is that the cheapest tier is always on and needs no network, and the expensive tier cannot break a turn.

Table 8: Observability tiers.

Tier	Cost	What it answers	Switch
Turn metrics	one log line, no network	How long, how many model calls and tool hops, how many tokens, which backend answered, and — when rates are configured — what it cost	always on
Langfuse traces	batched export on a background thread	What the turn looked like inside: every node, prompt, tool result, latency and token count, grouped by conversation	the key pair <i>is</i> the switch
scout doctor	local reads only	Whether <i>this checkout</i> can start, and what it will run with less of	a command
scout stats	reads the journal	The last <i>n</i> days of turns, aggregated	a command

The key pair is the switch. There is no separate on/off flag, because a flag can disagree with the credentials. Half a pair is reported as a warning by `scout doctor`, because it is the one state that reads as “on” in an environment file while tracing nothing. `tracing.REQUIRES` is the same tuple the doctor reads, so the report and the runtime cannot diverge.

Trace names are an API. `answer-slack-dm` and `run-job-digest` carry no user, agent or model. Langfuse dashboards, saved views and LLM-judge evaluators all target trace *names*, so anything per-run

in a name fragments all three; which agent ran and on which backend are a tag and a metadata field instead. For the same reason the turn owns its root span explicitly: trace-level input and output are read off that root, and they have to be the user’s question and the bot’s reply — not LangGraph’s message list, which answers “what is in the thread” rather than “what did it say”. `Turn.answered()` is how the reply gets there, including on the stuck path.

Reading the same names the SDK reads. Langfuse’s own client reads `LANGFUSE_BASE_URL` ahead of the host passed to its constructor. Reading only `LANGFUSE_HOST` therefore had scout doctor confidently reporting the EU default while traces went to the US endpoint. `_env_any` now reads the same names in the same order the SDK does, guarded by `test_env_any_reads_the_names_in_order`.

11 Testing strategy

The suite is 389 tests at 100 % statement *and* branch coverage, with `fail_under = 100` so the floor is the ceiling, running on Python 3.10 through 3.13 in CI with no credentials and no network access.

That is only possible because there are exactly two stub seams (Table 9). Chat models are scripted and installed into `scout.core.models` as if they were real backends, so the graph reaches them through the same `build/with_tools` path it uses for Claude. Every job source’s HTTP goes through `jobs/fetch`, so stubbing that one module stubs sixteen sources — which is the practical payoff of making them all share a fetch layer in the first place.

Table 9: What is faked, what is real, and what the suite refuses to do.

Concern	Approach	Why
Model providers	ScriptedModel registered as a real backend	The graph, the binding and the trimming are all exercised for real; only the network call is not
Job-board HTTP	stub	One seam covers every source, including the None contract
Slack	<code>jobs/fetch.requests</code> the adapter is tested against a fake say	DM filtering, commands and message splitting are logic, not I/O
Tracing	keys and <i>both</i> host names emptied in <code>conftest</code>	The SDK reads <code>LANGFUSE_BASE_URL</code> itself; one test was otherwise posting to Langfuse Cloud
LangSmith	<code>LANGSMITH_</code> <code>TRACING=false</code> pinned	Not Scout’s setting at all — <code>langsmith</code> ships with <code>langchain-core</code> and reads it, so a stale <code>.env</code> could export traces from a test run
Developer environment	settings pointed at <code>tmp_path</code>	A test that reads the author’s own <code>.env</code> or <code>data/</code> passes locally and fails in CI
Packaging	<code>test_packaging.py</code>	<code>requirements*.txt</code> are generated from <code>pyproject.toml</code> ; the Dockerfile installs from the flat list, so drift is a deployment bug
Documentation	<code>test_docs.py</code>	Every relative link and heading anchor in every Markdown file must resolve

Two habits do most of the work. First, the test names are sentences about behaviour (`test_every_board_being_down_is_not_reported_as_nothing_open`), so a failure report names the property that broke rather than the function that threw. Second, the invariants that are easy to break silently have a test each, named after the invariant — which is why the decision log above can cite one for nearly every row.

12 Deployment and operations

The whole platform is six systemd units on one free-tier instance (Figure 9), configured by a four-layer precedence chain (Table 10).

`deploy.sh` ships the working tree with `rsync` rather than doing a `git pull` on the box, for a specific reason: `.env` and `data/` are git-ignored but are exactly what the host needs, so one copy covers code,

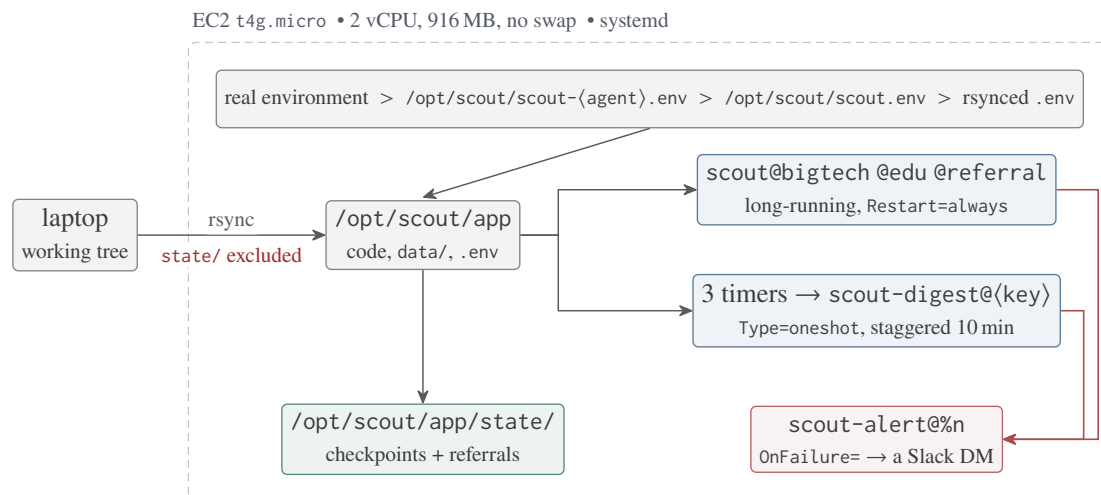


Figure 9: Deployment topology. Both service files are systemd *templates*: a second agent is `systemctl enable scout@edu`, not a second unit file. The precedence chain at the top is what lets host-local secrets outrank the rsynced `.env` without either file knowing about the other.

Table 10: Configuration surface. The rule is that anything which cannot be shared between agents goes in the per-agent file, and everything else stays in one place.

Layer	Holds	Precedence and reason
Process environment	AGENT, anything systemd sets	Highest. <code>load_dotenv</code> never overrides an existing key, so the deployed <code>EnvironmentFile</code> stays authoritative over a rsynced file
<code>.env.{agent}</code>	Slack token pair, CHECKPOINT_DB	Each agent is its own Slack app; each interactive agent needs its own database path
<code>.env</code>	API keys, limits, model names, Langfuse	Shared by every agent; never copied per agent
AgentSpec	prompt, tools, default backend, behaviour flags	Configuration that is code, because it is reviewed like code

résumé and secrets. The cost is that the box has no git history to identify itself with, so the script records the branch, the short revision and whether the tree was dirty into `/opt/scout/DEPLOYED`. Untracked files count as dirty, because rsync sends them and they then run on the box without appearing in any commit.

Two operational details are worth stating because they are the kind an interviewer probes. Socket Mode holds a websocket, and container platforms and systemd stop a process with `SIGTERM`, which by default kills Python outright — the socket is never closed and the shutdown path never runs. Scout installs a handler that raises `KeyboardInterrupt`, so `SIGTERM` unwinds through exactly the same path as `Ctrl-C`. And alerting goes to Slack rather than CloudWatch because the box has no IAM role; an alert that requires opening a console is an alert nobody reads. `Restart=always` means a single restart is normal and silent — systemd only reports failure once a unit exceeds its restart burst, which is the signal actually worth a notification. A bot that is running but has silently lost its websocket still looks healthy to systemd; the daily digest is the backstop for that, because if it stops arriving, something is wrong.

13 Limits, and what the next constraint would be

Table 11 is the list of things this design does not do, in the order in which they would actually start to hurt.

The honest summary is that the architecture is sized for its actual problem. The seams that exist — agent, tool, source, provider — are the four axes along which this system has in fact changed, repeatedly; there is no abstraction layer waiting for a second user, a second workspace, or a database that is not

Table 11: Known limits, ordered by how soon each would actually bite.

Limit	Why it exists	What would remove it
Single box, no redundancy	One t4g.micro; a reboot is downtime	Two instances behind nothing at all — Socket Mode means no load balancer, so it is an active/standby problem, not a routing one
Memory ceiling	916 MB, no swap, three resident processes	Staggering already covers the digests; a fourth interactive agent needs a bigger instance
Digest dedupe is prose-based	Agents return text, so the structured JobPosting objects are gone by reply time; the only dedupe is the model reading its own thread	Have the digest call the searchers directly and render in code, keeping the model for ranking only
No rate limiting toward boards	A referral sweep issues up to four concurrent requests and runs daily; nobody has complained	A token bucket in fetch, which is the single choke point every source already passes through
Company coverage is hand-maintained	directory.py is a table by design	Nothing — this is the cost of the completeness guarantee, and it is the right trade
Single user in practice	Per-user locks and per-user threads exist, but there is one Slack workspace and one résumé directory	Résumé storage per user, and a real secret store rather than environment files

SQLite, because none of those has been needed and each would have cost clarity now for flexibility later.

A Interview cheat sheet

1. **“Walk me through the architecture in two minutes.”** Slack DM adapter → ConversationalAgent → a LangGraph model/tool loop that every agent compiles to → tools, of which sixteen are job sources over a shared fetch/render/relevance layer. An agent is a declarative spec, history is a checkpointing thread per user, and everything else — the digest, the tracing, the fallback — hangs off those three facts.
2. **“Why LangGraph and not just a loop?”** I had the loop first. LangGraph bought checkpointing, subgraphs and a tool node, and it made tool traffic part of history so the model remembers what it looked up. The cost is that tool messages consume the context window, which is a trade I would make again.
3. **“How does a user switch models mid-conversation?”** History is stored as provider-agnostic LangChain message objects, not as any vendor’s wire format, so the thread replays into whichever integration is chosen. The backend rides in the graph config per turn, so one compiled graph serves every user on every model.
4. **“What happens when a job board is down?”** It returns None, never an empty list, and the reply names it as unchecked. That distinction is the whole reason the Referral Window’s answer can be trusted — “nothing open” where the truth is “I could not look” is what sends someone away from a company they should have asked.
5. **“Why not let the model figure out which board a company is on?”** It gets it right most of the time, and most of the time is exactly what a completeness claim cannot be. Routing in code is also what makes it possible to *name* the companies that have no board, instead of silently dropping them.
6. **“How do you stop it hallucinating a posting date?”** Two sources publish no date at all and several publish only relative wording. Those go into a separate `posted_label` field that is never coerced into a timestamp, undated postings sort last, and the digest prompt says to repeat the tool’s wording verbatim or write “no date given”.
7. **“What breaks if a turn runs out of tool calls?”** Without the repair, the user’s *next* message — because the thread would contain tool calls with no results, which providers reject. `_give_up()` writes a synthetic result for each abandoned call before recording the apology.
8. **“Where does the retry live, and why does that matter?”** Inside the model node. A node that raises commits nothing, so the retry starts clean while keeping tool results the turn already fetched. Retrying around the turn would either discard that work or hammer a dozen careers boards twice.
9. **“How do you keep an agent from doing something the user did not ask for?”** Capability, not instruction. Job agents import a read-only referral module, so the write tools are not in their schema at all. The one agent that may write has no tool to search off-list with. Both are pinned by tests named after the invariant.
10. **“How is it tested without network access?”** Two stub seams: chat models are scripted and registered as real backends, and every source’s HTTP goes through one fetch module. 389 tests, 100 % statement and branch coverage, Python 3.10–3.13, no credentials in CI.
11. **“What does observability cost you per turn?”** The always-on tier costs one log line and no network. Tracing is opt-in, built lazily on the first traced turn, and exported in batches on a background thread — and if it cannot start, it logs once and the process runs untraced. No branch in the graph knows either way.
12. **“What would you change?”** Render the digest in code from the structured postings rather than asking the model to reformat them — that would give real dedupe instead of relying on the model reading its own thread — and put a token bucket in the fetch layer before coverage grows further.

B The sixty-second version

Scout is a multi-agent job-search assistant that lives in Slack DMs. Each agent is a system prompt plus a set of tools, declared in one file and compiled onto a shared LangGraph model/tool loop; adding an agent never touches the graph. Job agents run behind a résumé parser whose output is cached in checkpointed graph state, so the résumé is parsed once per thread and every search is tailored without the user restating anything. Sixteen source modules cover twenty-nine companies over one shared HTTP, rendering and relevance layer, and each exposes its search twice — as a tool for the model, and as a plain function so a third agent can sweep every company where the user has a referral and say exactly which boards it could not reach. The whole thing runs as six systemd units on a free-tier EC2 instance, with one log line per turn always on, optional Langfuse tracing that cannot break a turn, and a test suite that runs at 100 % branch coverage with no network and no credentials.